

RAJALAKSHMI ENGINEERING COLLEGE
(Autonomous)
RAJALAKSHMI NAGAR, THANDALAM, CHENNAI-602105

EXPT NO: 7

DATE: _____

**A PYTHON PROGRAM TO IMPLEMENT UNCONSTRAINED OPTIMIZATION –
NONLINEAR LEAST SQUARES**

AIM:

To implement Nonlinear Least Squares optimization using `scipy.optimize.least_squares` to fit different mathematical models to noisy data.

PROCEDURE:

1. Define the nonlinear model function and the residual function (model - observed).
2. Generate synthetic data by adding Gaussian noise to the true model.
3. Choose appropriate initial parameter guess `x0`.
4. Call `least_squares(residual, x0, args=(t, y))` to fit the model.
5. Print and interpret the optimized parameter values.

PROBLEM 1: Fit a Power Law Model $y = a * t^b$ to noisy data

CODE

```
import numpy as np
from scipy.optimize import least_squares
def model(params, t): a, b = params; return a * np.power(t, b)
def residual(params, t, y): return model(params, t) - y
np.random.seed(10)
t = np.linspace(0.5, 5, 60)
true_params = [3.0, 0.7]
y = model(true_params, t) + 0.3 * np.random.randn(60)
x0 = [1.0, 1.0]
result = least_squares(residual, x0, args=(t, y))
print("True params : a=3.0, b=0.7")
print("Fitted params: a={:.4f}, b={:.4f}".format(result.x[0], result.x[1]))
print("Cost (residual norm):", round(result.cost, 4))
```

OUTPUT

```
True params : a=3.0, b=0.7
Fitted params: a=3.0212, b=0.6981
Cost (residual norm): 2.6153
```

PROBLEM 2: Fit a Polynomial $y = a + bx + cx^2$ using Nonlinear Least Squares

CODE

```
def model(params, t): a,b,c = params; return a + b*t + c*t**2
def residual(params, t, y): return model(params, t) - y
np.random.seed(77); t = np.linspace(-3, 3, 80)
```

```

true_params = [2.0, -1.5, 0.8]
y = model(true_params, t) + 0.5 * np.random.randn(80)
x0 = [0.0, 0.0, 0.0]
result = least_squares(residual, x0, args=(t, y))
a, b, c = result.x
print("True : a=2.0, b=-1.5, c=0.8")
print("Fitted : a={:.4f}, b={:.4f}, c={:.4f}".format(a, b, c))

```

OUTPUT

```

True : a=2.0, b=-1.5, c=0.8
Fitted : a=1.9872, b=-1.5063, c=0.8017

```

PROBLEM 3: Fit a Gaussian $y = A \cdot \exp(-(t-\mu)^2/(2\sigma^2))$ to peak data

CODE

```

def gaussian(params, t):
    A, mu, sigma = params
    return A * np.exp(-(t - mu)**2 / (2 * sigma**2))
def residual(params, t, y): return gaussian(params, t) - y
np.random.seed(55); t = np.linspace(-5, 5, 100)
true_params = [4.0, 1.0, 1.5]
y = gaussian(true_params, t) + 0.1 * np.random.randn(100)
x0 = [1.0, 0.0, 1.0]
result = least_squares(residual, x0, args=(t, y),
    bounds=([0, -np.inf, 0.01], np.inf))
A, mu, sigma = result.x
print("True : A=4.0, mu=1.0, sigma=1.5")
print("Fitted: A={:.4f}, mu={:.4f}, sigma={:.4f}".format(A, mu, sigma))

```

OUTPUT

```

True : A=4.0, mu=1.0, sigma=1.5
Fitted: A=3.9987, mu=1.0023, sigma=1.4978

```

RESULT:

Thus, Unconstrained Optimization using Nonlinear Least Squares has been implemented and executed successfully using Python.